

Databázy a vizualizácia

SQL - Príkaz JOIN

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

SQL - PRÍKAZ JOIN

Prečo potrebujeme JOIN

V databázach sú údaje často uložené vo viacerých tabuľkách. Jedna tabuľka obsahuje základné informácie o objektoch a ďalšie tabuľky obsahujú doplňujúce údaje. Príklad databázy filmov:

filmy

- id
- nazov
- rok

zanre

- id
- nazov

film_zaner

- film_id
- zaner_id

Ako zobrazíme názov filmu spolu s jeho žánrom (žánrami), keď sú údaje uložené v troch rôznych tabuľkách?

SQL - PRÍKAZ JOIN

Príklad údajov v tabuľkách

filmy		
id	nazov	rok
1	Avatar	2009
2	Matrix	1999

zanre	
id	nazov
1	Sci-Fi
2	Action

film_zaner	
film_id	zaner_id
1	1
1	2
2	1

Ako z týchto troch tabuliek získame výsledok? K riešeniu sa vrátíme na konci tejto časti prednášky.

Avatar – Sci-Fi

Avatar – Action

Matrix – Sci-Fi

SQL - PRÍKAZ JOIN

Príklad dvoch tabuliek

Predpokladajme databázu s informáciami o autách a ich farbách.

auta		
id	nazov	farba_id
1	BMW	1
2	Audi	2
3	Tesla	3
4	Hyundai	NULL
5	SKoda	1

farby	
id	nazov
1	modra
2	cervena
3	cierna
4	biela

Každé auto má priradenú práve jednu farbu pomocou identifikátora `farba_id`. Ako zobrazíme názov auta spolu s názvom jeho farby?

SQL - PRÍKAZ JOIN

Ľavá a pravá tabuľka a typy spojení

Pri zápise JOIN je dôležité poradie tabuliek.

```
SELECT * FROM auta  
LEFT/RIGHT/INNER JOIN farby  
ON auta.farba_id = farby.id
```

V tomto zápise je tabuľka auta ľavá tabuľka a tabuľka farby pravá tabuľka.

- **CROSS JOIN** – všetko so všetkým, kartézsky súčin.
- **INNER JOIN** – len riadky so zhodou v oboch tabuľkách, prienik.
- **LEFT JOIN** – všetky riadky z ľavej tabuľky a zhodné riadky z pravej.
- **RIGHT JOIN** – všetky riadky z pravej tabuľky a zhodné riadky z ľavej.

Ak zmeníme poradie tabuliek, zmení sa aj význam slov LEFT a RIGHT. Pri CROSS JOIN sa podmienka ON nepoužíva.

SQL - PRÍKAZ JOIN

CROSS JOIN – princíp

Predstavme si, že spojíme každé auto s každou farbou.

To znamená, že každý riadok tabuľky auta sa spojí so všetkými riadkami tabuľky farby.

Počet riadkov výsledku bude:

$$\text{počet áut} \times \text{počet farieb}$$

V našom príklade:

$$5 \text{ áut} \times 4 \text{ farby} = 20 \text{ riadkov}$$

Takéto spojenie sa nazýva **CROSS JOIN**.

SQL - PRÍKAZ JOIN

CROSS JOIN

```
SELECT a.nazov, f.nazov  
FROM auta a  
CROSS JOIN farby f;
```

Výsledok obsahuje všetky kombinácie riadkov z oboch tabuliek.

Príklad časti výsledku:

BMW – modra

BMW – cervena

BMW – cierna

BMW – biela

Audi – modra

Audi – cervena

...

SQL - PRÍKAZ JOIN

Od CROSS JOIN k spojeniu tabuliek

CROSS JOIN vytvára všetky možné kombinácie riadkov z oboch tabuliek.

Väčšinou však chceme spojiť len tie riadky, ktoré patria k sebe.

Preto definujeme **podmienku spojenia** medzi tabuľkami.

```
auta.farba_id = farby.id
```

Táto podmienka určuje, ktoré riadky z oboch tabuliek k sebe patria.

Spôsob, akým databáza pracuje s nezhodujúcimi sa riadkami, potom určuje konkrétny typ spojenia (napr. INNER JOIN, LEFT JOIN).

SQL - PRÍKAZ JOIN

INNER JOIN – princíp

INNER JOIN spája riadky dvoch tabuliek podľa zvolenej podmienky spojenia.

Do výsledku sa dostanú len tie riadky, pre ktoré existuje zhoda v oboch tabuľkách.

V našom príklade chceme spojiť auto s jeho farbou. SQL príkaz je:

```
SELECT auta.nazov AS auto, farby.nazov AS farba FROM auta  
INNER JOIN farby  
ON auta.farba_id = farby.id;
```

To znamená, že databáza spojí riadok tabuľky auta s tým riadkom tabuľky farby, ktorý má rovnaké identifikačné číslo farby.

Teoreticky by sme mohli použiť aj inú podmienku, napríklad

auta.id = farby.id

Takéto spojenie by SQL vykonal, ale nedávalo by logický zmysel, pretože identifikátor auta nesúvisí s identifikátorom farby.

SQL - PRÍKAZ JOIN

INNER JOIN – príklad

Výsledkom bude tabuľka obsahujúca názov auta a názov jeho farby.

auto	farba
BMW	modra
Audi	cervena
Tesla	cierna
Skoda	modra

Auto Hyundai sa vo výsledku neobjaví, pretože nemá priradenú farbu (`farba_id = NULL`), takže preň neexistuje zhoda v tabuľke farby. Takisto sa vo výslednej tabuľke neobjaví biela farba lebo sa nevyskytuje v stĺpci `farba_id` tabuľky auta.

SQL - PRÍKAZ JOIN

LEFT JOIN – princíp

LEFT JOIN spája riadky dvoch tabuliek podľa zvolenej podmienky spojenia podobne ako INNER JOIN. Rozdiel je v tom, že pri LEFT JOIN sa vo výsledku zachovávajú všetky riadky z ľavej tabuľky.

Ak pre niektorý riadok z ľavej tabuľky neexistuje zodpovedajúci riadok v pravej tabuľke, hodnoty stĺpcov z pravej tabuľky budú NULL.

V našom prípade je ľavou tabuľkou auta a pravou tabuľkou farby.

```
SELECT a.nazov AS auto, f.nazov AS farba  
FROM auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id;
```

Všetky autá sa teda zobrazia vo výsledku, aj keď pre ne neexistuje priradená farba.

SQL - PRÍKAZ JOIN

LEFT JOIN – príklad

Výsledok dotazu môže vyzeráť napríklad takto:

auto	farba
BMW	modra
Audi	cervena
Tesla	cierna
Hyundai	NULL
Skoda	modra

Auto Hyundai sa vo výsledku zobrazí, aj keď nemá priradenú farbu. Preto sa v stĺpci farba objaví NULL.

SQL - PRÍKAZ JOIN

LEFT JOIN a IS NULL

Pomocou LEFT JOIN môžeme nájsť záznamy, ktoré nemajú zodpovedajúci riadok v druhej tabuľke.

V našom príklade chceme nájsť autá, ktoré nemajú priradenú farbu.

```
SELECT a.nazov AS auto  
FROM auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id  
WHERE f.id IS NULL;
```

Výsledok:

auto
Hyundai

SQL - PRÍKAZ JOIN

LEFT JOIN a IS NOT NULL

Ak z výsledku LEFT JOIN odstránime riadky s hodnotou NULL, zostanú len tie riadky, ktoré majú zodpovedajúci riadok v oboch tabuľkách.

```
SELECT a.nazov AS auto, f.nazov AS farba  
FROM auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id  
WHERE f.id IS NOT NULL;
```

Výsledok je rovnaký ako pri použití INNER JOIN.

SQL - PRÍKAZ JOIN

RIGHT JOIN – princíp

RIGHT JOIN funguje podobne ako LEFT JOIN, ale zachováva všetky riadky z pravej tabuľky.

Ak pre niektorý riadok z pravej tabuľky neexistuje zodpovedajúci riadok v ľavej tabuľke, hodnoty stĺpcov z ľavej tabuľky budú NULL.

V našom prípade je pravou tabuľkou farby.

```
SELECT a.nazov AS auto, f.nazov AS farba  
FROM auta a  
RIGHT JOIN farby f  
ON a.farba_id = f.id;
```

SQL - PRÍKAZ JOIN

RIGHT JOIN – príklad

Výsledok dotazu môže vyzerat napríklad takto:

auto	farba
BMW	modra
Skoda	modra
Audi	cervena
Tesla	cierna
NULL	biela

Farba biela sa zobrazí aj keď pre ňu neexistuje žiadne auto. Preto sa v stĺpci auto objaví hodnota NULL.

SQL - PRÍKAZ JOIN

LEFT JOIN a RIGHT JOIN

Pri spojení dvoch tabuliek je RIGHT JOIN len zrkadlový zápis LEFT JOIN.

Napríklad:

```
SELECT auta.nazov AS auto, farby.nazov AS farba  
FROM auta  
LEFT JOIN farby  
ON auta.farba_id = farby.id;
```

je ekvivalentné zápisu

```
SELECT auta.nazov AS auto, farby.nazov AS farba  
FROM farby  
RIGHT JOIN auta  
ON auta.farba_id = farby.id;
```

Rozdiel je len v poradí tabuliek.

SQL - PRÍKAZ JOIN

RIGHT JOIN a IS NULL

Pomocou RIGHT JOIN môžeme nájsť farby, ktoré nie sú priradené žiadnemu autu.

```
SELECT f.nazov  
FROM auta a  
RIGHT JOIN farby f  
ON a.farba_id = f.id  
WHERE a.id IS NULL;
```

Výsledok:

farba

biela

SQL - PRÍKAZ JOIN

RIGHT JOIN a IS NOT NULL

Ak odstránime riadky s NULL, zostanú len tie, ktoré majú zodpovedajúci riadok v oboch tabuľkách.

```
SELECT a.nazov, f.nazov  
FROM auta a  
RIGHT JOIN farby f  
ON a.farba_id = f.id  
WHERE a.id IS NOT NULL;
```

Výsledok je rovnaký ako pri použití INNER JOIN.

SQL - PRÍKAZ UNION

UNION. Spojenie výsledkov dotazov

Niekedy chceme spojiť výsledky viacerých SQL dotazov do jednej tabuľky.

Na to slúži príkaz **UNION**.

Všeobecný zápis:

```
SELECT ...  
FROM ...
```

```
UNION
```

```
SELECT ...  
FROM ...
```

Výsledkom je jedna tabuľka obsahujúca riadky z oboch dotazov.

Podmienka: oba dotazy musia mať rovnaký počet stĺpcov a kompatibilné dátové typy.

SQL - PRÍKAZ UNION

UNION a UNION ALL

Existujú dve varianty spojenia výsledkov.

UNION

- spojí výsledky dvoch dotazov
- odstráni duplicitné riadky

UNION ALL

- spojí výsledky dvoch dotazov
- duplicity ponechá

Príklad:

```
SELECT nazov FROM auta  
UNION  
SELECT nazov FROM farby;
```

Tento príklad je len ilustračný. V praxi sa zvyčajne spájajú výsledky podobných dotazov.

SQL - PRÍKAZ JOIN

FULL OUTER JOIN v MySQL

Niektoré databázy podporujú príkaz **FULL OUTER JOIN**, ktorý vráti všetky riadky z oboch tabuliek. MySQL tento príkaz priamo nepodporuje.

Rovnaký výsledok môžeme vytvoriť pomocou UNION.

```
SELECT a.nazov, f.nazov FROM auta a
LEFT JOIN farby f
ON a.farba_id = f.id
UNION
SELECT a.nazov, f.nazov FROM auta a
RIGHT JOIN farby f
ON a.farba_id = f.id;
```

SQL - PRÍKAZ JOIN

Exclusive join

Niekedy chceme nájsť riadky, ktoré sa nachádzajú len v jednej tabuľke, ale nie v oboch.

Takéto spojenie sa niekedy nazýva **exclusive join** alebo **symetrický rozdiel množín**.

Výsledok obsahuje:

- riadky z ľavej tabuľky bez zhody,
- riadky z pravej tabuľky bez zhody.

SQL na to nemá samostatný príkaz.

SQL - PRÍKAZ JOIN

Exclusive join – príklad

Exclusive join môžeme vytvoriť spojením dvoch dotazov.

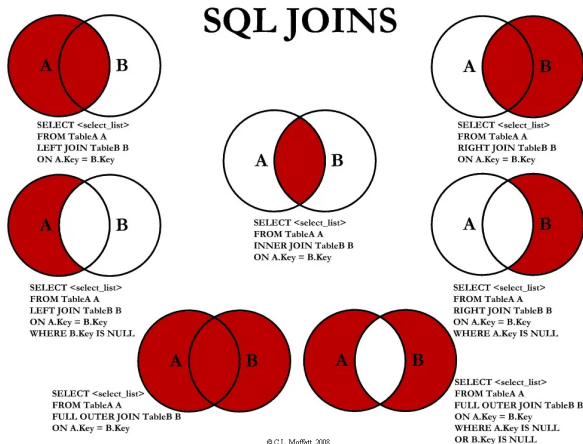
```
SELECT a.nazov, f.nazov FROM auta a
LEFT JOIN farby f
ON a.farba_id = f.id
WHERE f.id IS NULL
UNION
SELECT a.nazov, f.nazov FROM auta a
RIGHT JOIN farby f
ON a.farba_id = f.id
WHERE a.id IS NULL;
```

Výsledok obsahuje:

- autá bez farby,
- farby bez auta.

SQL - PRÍKAZ JOIN

Sumarizácia joinov



Obrázok ilustruje rôzne spôsoby spojenia dvoch tabuliek v SQL.

SQL - PRÍKAZ JOIN

Ktorý JOIN použiť

- **INNER JOIN** – ak chceme zobrazíť len riadky, ktoré majú zhodu v oboch tabuľkách.
- **LEFT JOIN** – ak chceme všetky riadky z ľavej tabuľky a k nim zhodné riadky z pravej tabuľky.
- **RIGHT JOIN** – ak chceme všetky riadky z pravej tabuľky a k nim zhodné riadky z ľavej tabuľky.
- **CROSS JOIN** – ak chceme všetky kombinácie riadkov z oboch tabuliek.

Pri **LEFT JOIN** a **RIGHT JOIN** môžu vo výsledku vzniknúť hodnoty **NULL**, ak pre niektorý riadok neexistuje zodpovedajúci riadok v druhej tabuľke.

SQL - PRÍKAZ JOIN

JOIN a filtrovanie pomocou NULL

- **LEFT JOIN** – všetky riadky z ľavej tabuľky.
- **LEFT JOIN + WHERE pravá.id IS NULL** – riadky z ľavej tabuľky, ktoré nemajú pár v pravej tabuľke.
- **LEFT JOIN + WHERE pravá.id IS NOT NULL** – riadky z ľavej tabuľky, ktoré majú pár v pravej tabuľke. Výsledok je zvyčajne rovnaký ako pri **INNER JOIN**.
- **RIGHT JOIN** – všetky riadky z pravej tabuľky.
- **RIGHT JOIN + WHERE ľavá.id IS NULL** – riadky z pravej tabuľky, ktoré nemajú pár v ľavej tabuľke.
- **RIGHT JOIN + WHERE ľavá.id IS NOT NULL** – riadky z pravej tabuľky, ktoré majú pár v ľavej tabuľke. Výsledok je zvyčajne rovnaký ako pri **INNER JOIN**.

SQL - PRÍKAZ JOIN

JOIN v kombinácii s UNION

- **LEFT JOIN UNION RIGHT JOIN** – všetky riadky z oboch tabuliek, teda náhrada za **FULL OUTER JOIN** v MySQL.
- **LEFT JOIN + IS NULL UNION RIGHT JOIN + IS NULL** – len riadky bez páru v druhej tabuľke, teda symetrický rozdiel.
- **LEFT JOIN + IS NOT NULL UNION RIGHT JOIN + IS NOT NULL** – len riadky, ktoré majú pár v druhej tabuľke. Výsledok je zvyčajne rovnaký ako pri **INNER JOIN**.
- **UNION** – spojí výsledky príkazov a odstráni duplicitné riadky.
- **UNION ALL** – spojí výsledky príkazov a duplicity ponechá.

Pri **FULL OUTER JOIN** sa zvyčajne používa **UNION**, aby sa zhodné riadky nezobrazili dvakrát.

SQL - PRÍKAZ JOIN

SELF JOIN – príklad tabuľky

Predpokladajme tabuľku zamestnancov. Každý zamestnanec môže mať nadriadeného.

zamestnanci

id	meno	sef_id
1	Jana	NULL
2	Peter	1
3	Martin	1

Stĺpec `sef_id` obsahuje identifikátor nadriadeného zamestnanca.

SQL - PRÍKAZ JOIN

SELF JOIN – spojenie tabuľky samej so sebou

Niekedy potrebujeme spojiť tabuľku samu so sebou. Takéto spojenie sa nazýva **SELF JOIN**.

V našom prípade chceme zobraziť zamestnanca a meno jeho nadriadeného.

```
SELECT z.meno AS zamestnanec,  
         s.meno AS sef  
FROM zamestnanci z  
LEFT JOIN zamestnanci s  
ON z.sef_id = s.id;
```

Používame aliasy z a s, aby databáza vedela rozlíšiť, na ktorý výskyt tabuľky zamestnanci sa odkazujeme.

SQL - PRÍKAZ JOIN

Príklad viacerých tabuliek

Predpokladajme databázu filmov a žánrov.

filmy	
id	nazov
1	Matrix
2	Titanic
3	Avatar

zanre	
id	nazov
1	sci-fi
2	drama
3	romanticky

film_zaner	
film_id	zaner_id
1	1
2	2
2	3
3	1

Ako zobrazíme názov filmu a názov jeho žánru v jednej tabuľke?

SQL - PRÍKAZ JOIN

JOIN troch tabuliek

```
SELECT f.nazov AS film, z.nazov AS zaner  
FROM filmy f  
INNER JOIN film_zaner fz  
ON f.id = fz.film_id  
INNER JOIN zane z  
ON fz.zaner_id = z.id;
```

Postup spojenia:

- tabuľka filmy sa spojí s tabuľkou film_zaner,
- následne sa výsledok spojí s tabuľkou zane.

Takto získame názov filmu a názov jeho žánru.

SQL - PRÍKAZ JOIN

Alternatívny zápis 1

Dotaz môžeme začať aj od prepojuvacej tabuľky.

```
SELECT f.nazov AS film, z.nazov AS zaner  
FROM film_zaner fz  
INNER JOIN filmy f  
ON fz.film_id = f.id  
INNER JOIN zane z  
ON fz.zaner_id = z.id;
```

Výsledok bude rovnaký. Dôležité je len správne určiť podmienky spojenia.

SQL - PRÍKAZ JOIN

Alternatívny zápis 2

```
SELECT f.nazov AS film, z.nazov AS zaner  
FROM filmy f  
LEFT JOIN film_zaner fz  
ON f.id = fz.film_id  
INNER JOIN zane z  
ON fz.zaner_id = z.id;
```

Aj keď používame LEFT JOIN, následný INNER JOIN odstráni riadky bez zodpovedajúceho žánru. Filmy bez žánru sa preto vo výsledku neobjavia.

SQL - PRÍKAZ JOIN

Alternatívny zápis 3

```
SELECT f.nazov AS film, z.nazov AS zaner  
FROM filmy f  
LEFT JOIN film_zaner fz  
ON f.id = fz.film_id  
LEFT JOIN zanre z  
ON fz.zaner_id = z.id;
```

V tomto prípade používame dva LEFT JOIN.

Výsledok obsahuje všetky filmy. Ak film nemá priradený žáner, v stĺpci zaner sa zobrazí hodnota NULL.

Následne záleží na tom, či chceme tieto riadky vo výsledku ponechať alebo odstrániť. Ak chceme zobrazíť len filmy s existujúcim žánrom, môžeme použiť napríklad podmienku WHERE z.id IS NOT NULL

SQL - PRÍKAZ JOIN

Alternatívny zápis 4

```
SELECT f.nazov AS film, z.nazov AS zaner  
FROM zane z  
RIGHT JOIN film_zaner fz  
ON z.id = fz.zaner_id  
RIGHT JOIN filmy f  
ON fz.film_id = f.id;
```

Tento zápis je zrkadlovou verziou predchádzajúceho dotazu a zachová všetky filmy.

Ak film nemá priradený žáner, v stĺpci zaner sa zobrazí hodnota NULL.

Následne záleží na tom, či chceme tieto riadky vo výsledku ponechať alebo odstrániť, napríklad pomocou podmienky `WHERE z.id IS NOT NULL`

Pri spojení viacerých tabuliek môžeme začať od ľubovoľnej tabuľky.

Dôležité je:

- vedieť, ktoré stĺpce patria k sebe,
- správne definovať podmienky v klauzule ON.

Ak chceme vo výsledku len riadky, ktoré majú zhodu vo všetkých tabuľkách, môžeme použiť **INNER JOIN**. V takom prípade na poradí tabuliek zvyčajne nezáleží.

V praxi sa často používa prepojovacia tabuľka pri reláciách typu **many-to-many**.

SQL - PRÍKAZ JOIN

Vnorený príkaz vs JOIN. Príklad – autá a farby

Predpokladajme nasledujúce tabuľky.

auta		
id	nazov	farba_id
1	BMW	1
2	Audi	2
3	Tesla	3
4	Hyundai	NULL
5	Skoda	1

farby	
id	nazov
1	modra
2	cervena
3	cierna
4	biela

Ako zobrazíme názvy áut, ktoré majú farbu **modra**?

SQL - PRÍKAZ JOIN

Vnorený príkaz vs JOIN. Riešenie pomocou vnoreného dotazu

Najprv nájdeme identifikátor farby modra a potom vyberieme autá s týmto identifikátorom.

```
SELECT nazov  
FROM auta  
WHERE farba_id = (  
    SELECT id  
    FROM farby  
    WHERE nazov = 'modra'  
);
```

Vnorený dotaz najprv nájde id farby modra. Nadradený dotaz potom zobrazí autá s týmto identifikátorom.

SQL - PRÍKAZ JOIN

Vnorený príkaz vs JOIN. Riešenie pomocou INNER JOIN

Rovnaký problém môžeme vyriešiť aj pomocou spojenia tabuliek.

```
SELECT a.nazov  
FROM auta a  
INNER JOIN farby f  
ON a.farba_id = f.id  
WHERE f.nazov = 'modra';
```

Tabuľky auta a farby sa najprv spoja podľa identifikátora farby a potom sa vo výsledku vyberú len riadky, kde názov farby je modra.

V praxi databázový systém často dokáže optimalizovať oba prístupy (vnorený dotaz aj JOIN) na podobný vykonávací plán. Pri práci s viacerými tabuľkami sa však zvyčajne odporúča používať JOIN, pretože je prehľadnejší a ľahšie sa rozširuje o ďalšie tabuľky.

SQL - PRÍKAZ JOIN

Riešenie pomocou vnoreného dotazu

Ktoré farby nie sú priradené k žiadnemu autu?

```
SELECT nazov  
FROM farby  
WHERE id NOT IN (  
    SELECT farba_id  
    FROM auta  
    WHERE farba_id IS NOT NULL  
);
```

Vnorený dotaz najprv vráti identifikátory farieb, ktoré sa používajú v tabuľke auta.

Nadradený dotaz potom zobrazí tie farby, ktorých identifikátor sa v tomto zozname nenachádza.

SQL - PRÍKAZ JOIN

Riešenie pomocou JOIN

```
SELECT f.nazov  
FROM farby f  
LEFT JOIN auta a  
ON f.id = a.farba_id  
WHERE a.id IS NULL;
```

Tabuľka farby sa spojí s tabuľkou auta. Používame LEFT JOIN, nie INNER JOIN, pretože chceme zachovať všetky farby. Ak pre niektorú farbu neexistuje zodpovedajúce auto, stĺpce z tabuľky auta budú mať NULL.

Podmienka a.id IS NULL vyberie len tie farby, ktoré nie sú priradené k žiadnemu autu.

SQL - kombinácia príkazov

JOIN + GROUP BY

V praxi sa SQL príkazy často kombinujú. Napríklad spojenie tabuliek pomocou JOIN spolu s agregáciou.

Napríklad chceme zistiť koľko žánrov má každý film.

```
SELECT f.nazov, COUNT(fz.zaner_id) AS pocet_zanrov  
FROM filmy f  
LEFT JOIN film_zaner fz  
ON f.id = fz.film_id  
GROUP BY f.id, f.nazov;
```

Dotaz najprv spojí tabuľky filmy a film_zaner. Potom príkaz GROUP BY vytvorí skupiny podľa filmu a funkcia COUNT() spočíta počet žánrov pre každý film.

Pripájanie tabuľky zane by bolo zbytočné, pretože počet žánrov vieme zistiť priamo z prepovacej tabuľky film_zaner.

SQL - Aktualizácia údajov pomocou JOIN

Pridanie názvu farby do tabuľky áut

Predpokladajme, že do tabuľky auta chceme pridať aj názov farby.

Najprv pridáme nový stĺpec.

```
ALTER TABLE auta  
ADD COLUMN farba_nazov VARCHAR(50);
```

Následne môžeme stĺpec vyplniť pomocou spojenia tabuliek.

```
UPDATE auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id  
SET a.farba_nazov = f.nazov;
```

Tabuľky sa najprv spoja rovnako ako pri príkaze SELECT a príkaz SET zapíše názov farby do tabuľky auta.